Machine Learning Assignment 02

Loss Function Analysis: Comparing MSE, MAE, and Huber Loss

Student Name: Abu Bakar
Student ID: NUM-BSCS-2022-41
Course: Machine Learning
Date: December 03, 2025

1. Introduction

In regression analysis, the choice of loss function significantly impacts model performance, especially in the presence of outliers. This report investigates three popular loss functions: Mean Squared Error (MSE), Mean Absolute Error (MAE), and Huber Loss. We analyze their behavior on both clean and noisy datasets to understand their strengths, weaknesses, and appropriate use cases.

2. Objective

The primary objective of this assignment is to:

- Understand how different loss functions behave with clean vs. noisy data
- Implement MSE, MAE, and Huber regression from scratch and using scikit-learn
- Visualize and compare the regression lines produced by each method
- Analyze the robustness of each loss function to outliers
- Provide recommendations for choosing appropriate loss functions

3. Datasets

Two datasets were used in this analysis:

3.1 Clean Dataset

- Number of samples: 80
- Experience range: 0.1 - 19.7 years
- Salary range: 22.6 - 97.6k USD
- Characteristics: Linear relationship with minimal noise

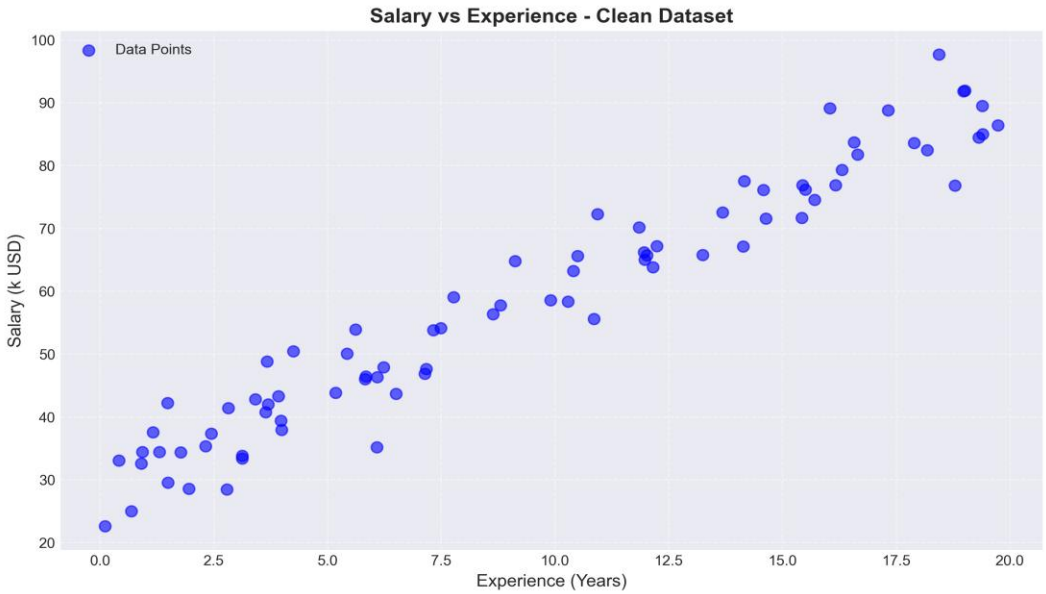
3.2 Dataset with Outliers

- Number of samples: 88
- Experience range: 0.1 - 19.7 years
- Salary range: 3.4 - 240.0k USD
- Characteristics: Contains extreme outliers that deviate from the main trend

4. Task 1: Data Exploration

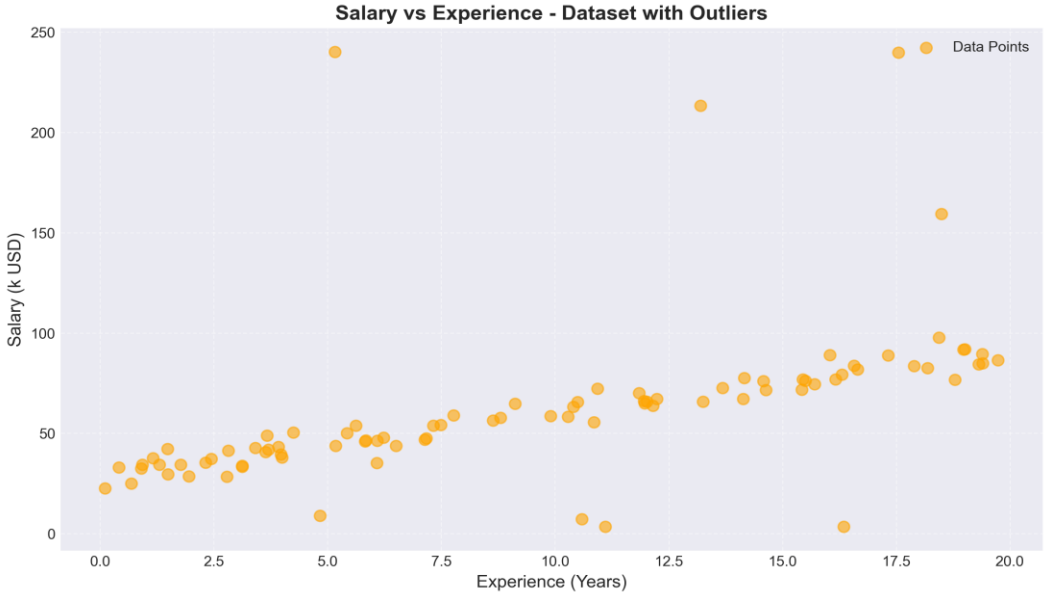
4.1 Clean Dataset Exploration

The clean dataset shows a clear linear relationship between years of experience and salary. Visual inspection reveals no significant outliers or anomalies.



4.2 Dataset with Outliers Exploration

The outlier dataset contains several extreme values that deviate significantly from the main trend. These outliers will test the robustness of different loss functions.



5. Task 2: MSE Regression (Ordinary Least Squares)

Mean Squared Error (MSE) is the most commonly used loss function in regression. It minimizes the sum of squared differences between predicted and actual values:

$$L(y, \hat{y}) = (1/n)\ \Sigma(y_i - \hat{y}_i)^2$$

5.1 Results on Clean Dataset

- Equation: y = 3.11x + 28.91
- MSE: 22.47
- MAE: 3.70

5.2 Results on Dataset with Outliers

- Equation: y = 3.44x + 29.64
- MSE: 1159.86
- MAE: 13.75

6. Task 3: MAE Regression (Least Absolute Deviations)

Mean Absolute Error (MAE) minimizes the sum of absolute differences. It is more robust to outliers than MSE:

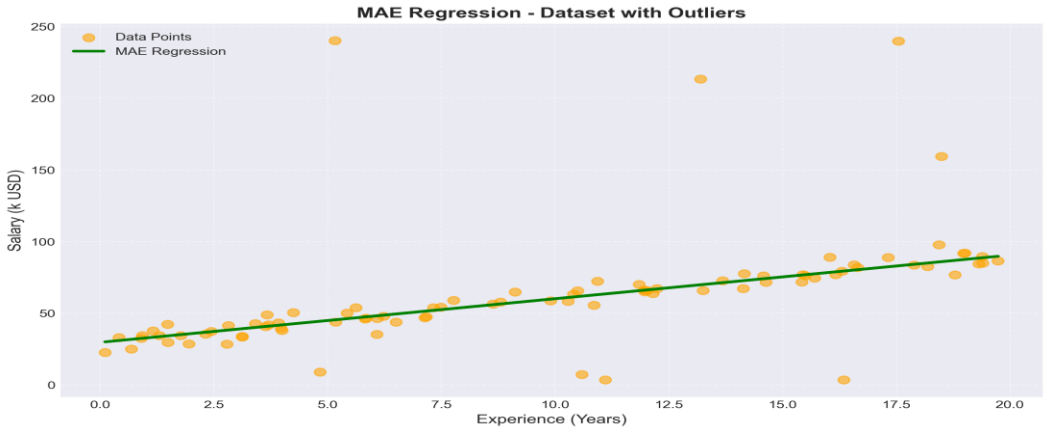
$$L(y, \hat{y}) = (1/n)\ \Sigma|y_i - \hat{y}_i|$$

6.1 Results on Clean Dataset

- Equation: y = 3.04x + 29.67
- MSE: 22.66
- MAE: 3.68

6.2 Results on Dataset with Outliers

- Equation: y = 3.04x + 29.66
- MSE: 1180.11
- MAE: 12.37



7. Task 4: Huber Regression

Huber Loss combines the best of MSE and MAE. It behaves like MSE for small errors (≤ δ = 1.35) and like MAE for large errors:

$$L_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| \leq \delta \\ \delta|y - \hat{y}| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases}$$

7.1 Results on Clean Dataset

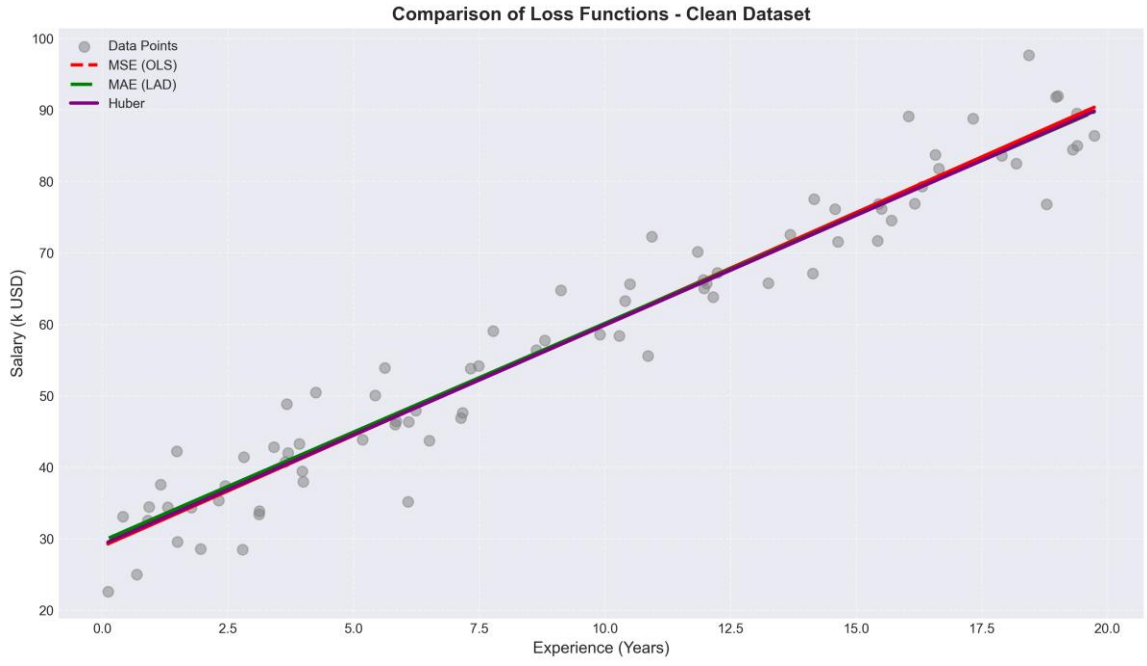
- Equation: y = 3.07x + 29.15
- MSE: 22.54
- MAE: 3.69
- Epsilon (δ): 1.35

7.2 Results on Dataset with Outliers

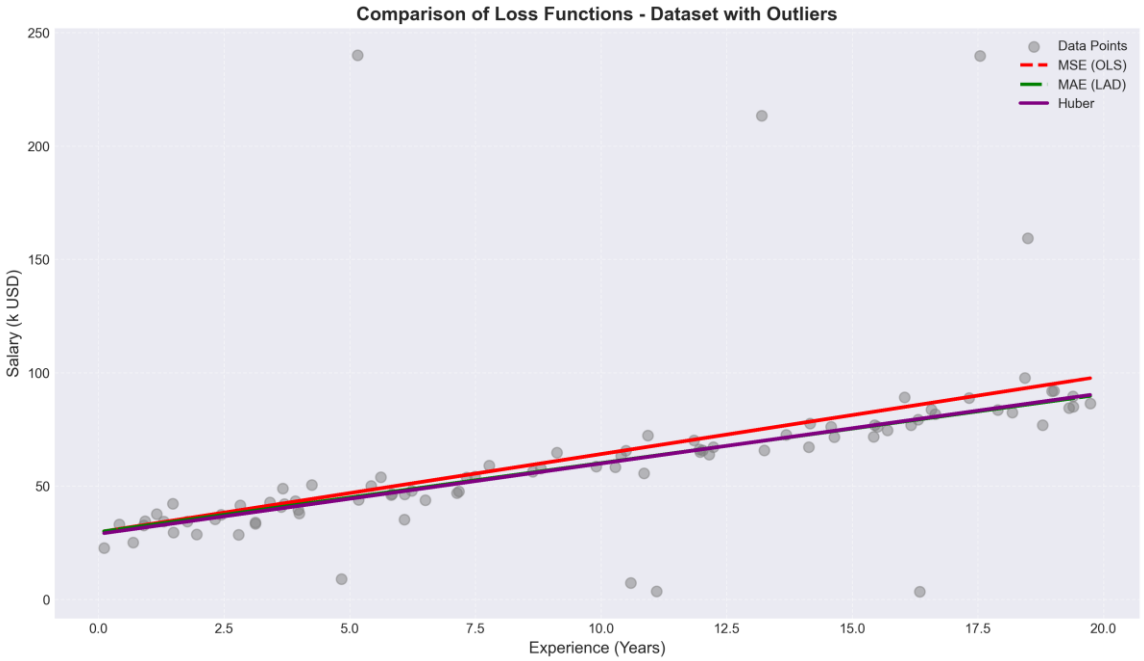
- Equation: y = 3.11x + 28.80
- MSE: 1179.97
- MAE: 12.38
- Epsilon (δ): 1.35

7.3 Visual Comparison of All Methods

Clean Dataset:



Dataset with Outliers:



8. Task 5: Comparative Analysis Report

Dataset	Method	Slope	MSE	MAE
Clean	MSE	3.11	22.47	3.70
Clean	MAE	3.04	22.66	3.68
Clean	Huber	3.07	22.54	3.69
Outliers	MSE	3.44	1159.86	13.75
Outliers	MAE	3.04	1180.11	12.37
Outliers	HUBER	3.11	1179.97	12.38

8.2 Effects of Outliers

Clean Dataset:

- Slope difference (MSE vs MAE): 0.0713
- All three methods produce nearly identical results
- No outliers present to influence the models

Dataset with Outliers:

- Slope difference (MSE vs MAE): 0.3994

8.3 Strengths and Weaknesses

MSE (Mean Squared Error):

Strengths:

- Computationally efficient with closed-form solution
- Differentiable everywhere
- Optimal for normally distributed errors
- Provides Maximum Likelihood Estimation for Gaussian noise

Weaknesses:

- Highly sensitive to outliers due to squared error term
- Can produce misleading results with noisy data
- Penalizes large errors disproportionately

MAE (Mean Absolute Error):

Strengths:

- Highly robust to outliers
- Treats all errors equally (linear penalty)
- Represents median trend rather than mean
- Excellent for noisy or contaminated data

Weaknesses:

- Computationally expensive (requires iterative optimization)
- Not differentiable at zero
- May be slower to converge

Huber Loss:

Strengths:

- Balanced approach combining MSE and MAE benefits
- Robust to outliers while maintaining differentiability
- Smooth transition between quadratic and linear loss
- Versatile for various data conditions

Weaknesses:

- Requires tuning of epsilon (δ) parameter
- Slightly more complex than MSE or MAE alone

8.4 Recommendations

For Clean Dataset:

- Recommended: MSE (Linear Regression)
- Reason: Most efficient and optimal for normally distributed errors

- All methods perform similarly, but MSE is simplest

For Dataset with Outliers:

- Recommended: MAE or Huber Regression
- Reason: Robustness to outliers is critical
- MSE fails to capture the true underlying relationship
- Huber offers good balance if differentiability is needed

9. Conclusion

This assignment successfully demonstrated the critical importance of loss function selection in regression analysis. Through empirical analysis of both clean and noisy datasets, we observed:

- MSE is optimal for clean data with normally distributed errors but fails catastrophically in the presence of outliers
- MAE provides exceptional robustness to outliers by treating all errors linearly, making it ideal for noisy or contaminated datasets
- Huber Loss offers a practical compromise, combining the efficiency of MSE for small errors with the robustness of MAE for large errors

The choice of loss function should be guided by data characteristics, particularly the presence and impact of outliers. For production systems, Huber Loss often provides the best balance between performance and robustness.

10. References

- Scikit-learn Documentation: Linear Models - https://scikitlearn.org/stable/modules/linear_model.html
- Huber, P. J. (1964). "Robust Estimation of a Location Parameter"
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). "An Introduction to Statistical Learning"
- Course Materials: Machine Learning - Loss Functions

11. Code

1. Code for Clean Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, HuberRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error
from scipy.optimize import minimize
```

```
# --- Configuration ---
FILE_PATH = "salary_experience_clean.csv"
DATASET_NAME = "Clean Dataset"

# --- Task 1: Data Exploration ---
print(f"--- Task 1: Data Exploration ({DATASET_NAME}) ---")
# Load dataset
df = pd.read_csv(FILE_PATH)
X = df['experience_years'].values
y = df['salary_k'].values

# Reshape X for sklearn (needs 2D array)
X_reshaped = X.reshape(-1, 1)

# Create scatter plot
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='blue', label='Data Points', alpha=0.6)
plt.title(f"Salary vs Experience ({DATASET_NAME})")
plt.xlabel("Experience (Years)")
plt.ylabel("Salary (k)")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
plt.savefig('scatter_plot.png')
plt.close()
print("Scatter plot created. The data shows a clear linear trend with no visible outliers.\n")

# --- Task 2: Implement MSE Regression (Ordinary Least Squares) ---
print(f"--- Task 2: MSE Regression ({DATASET_NAME}) ---")
# Fit Linear Regression (minimizes MSE)
mse_model = LinearRegression()
mse_model.fit(X_reshaped, y)
y_pred_mse = mse_model.predict(X_reshaped)

# Calculate metrics
mse_mse = mean_squared_error(y, y_pred_mse)
mae_mse = mean_absolute_error(y, y_pred_mse)

print(f"MSE Model Coefficients: Slope={mse_model.coef_[0]:.2f},
Intercept={mse_model.intercept_:.2f}")
print(f"MSE (Mean Squared Error): {mse_mse:.2f}")
print(f"MAE (Mean Absolute Error): {mae_mse:.2f}")

# Plot MSE Regression
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='blue', label='Data Points', alpha=0.6)
plt.plot(X, y_pred_mse, color='red', linewidth=2, label='MSE Regression (OLS)')
```

```
plt.title(f"MSE Regression on {DATASET_NAME}")
plt.xlabel("Experience (Years)")
plt.ylabel("Salary (k)")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
plt.savefig('mse_regression.png')
plt.close()
print("\n")

# --- Task 3: Implement MAE Regression (Least Absolute Deviations) ---
print(f"--- Task 3: MAE Regression ({DATASET_NAME}) ---")

# Define MAE loss function for optimization
def mae_loss_func(params, X_vals, y_vals):
    m, c = params
    y_pred = m * X_vals + c
    return np.sum(np.abs(y_vals - y_pred))

# Optimize to find best m (slope) and c (intercept)
initial_guess = [1, 1]
result = minimize(mae_loss_func, initial_guess, args=(X, y))
m_mae, c_mae = result.x
y_pred_mae = m_mae * X + c_mae

# Calculate metrics for MAE model
mse_mae_model = mean_squared_error(y, y_pred_mae)
mae_mae_model = mean_absolute_error(y, y_pred_mae)

print(f"MAE Model Coefficients: Slope={m_mae:.2f}, Intercept={c_mae:.2f}")
print(f"MSE (Mean Squared Error): {mse_mae_model:.2f}")
print(f"MAE (Mean Absolute Error): {mae_mae_model:.2f}")

# Plot MAE Regression
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='blue', label='Data Points', alpha=0.6)
plt.plot(X, y_pred_mae, color='green', linewidth=2, label='MAE Regression (LAD)')
plt.title(f"MAE Regression on {DATASET_NAME}")
plt.xlabel("Experience (Years)")
plt.ylabel("Salary (k)")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
plt.savefig('mae_regression.png')
plt.close()
print("\n")

# --- Task 4: Implement Huber Regression ---
```

```
print(f"--- Task 4: Huber Regression ({DATASET_NAME}) ---")
# Fit Huber Regressor
# epsilon=1.35 is standard; it determines the threshold between MSE and MAE behavior
huber_model = HuberRegressor(epsilon=1.35)
huber_model.fit(X_reshaped, y)
y_pred_huber = huber_model.predict(X_reshaped)

# Calculate metrics
mse_huber = mean_squared_error(y, y_pred_huber)
mae_huber = mean_absolute_error(y, y_pred_huber)

print(f"Huber Model Coefficients: Slope={huber_model.coef_[0]:.2f},
Intercept={huber_model.intercept_:.2f}")
print(f"MSE (Mean Squared Error): {mse_huber:.2f}")
print(f"MAE (Mean Absolute Error): {mae_huber:.2f}")

# Plot All Comparisons
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='gray', label='Data Points', alpha=0.5)
plt.plot(X, y_pred_mse, color='red', linestyle='--', linewidth=2, label='MSE (OLS)')
plt.plot(X, y_pred_mae, color='green', linestyle='-.', linewidth=2, label='MAE (LAD)')
plt.plot(X, y_pred_huber, color='purple', linestyle='-', linewidth=2, label='Huber')
plt.title(f"Comparison of Loss Functions on {DATASET_NAME}")
plt.xlabel("Experience (Years)")
plt.ylabel("Salary (k)")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
plt.savefig('huber_regression.png')
plt.close()
print("\n")

# --- Task 5: Comparative Analysis Report ---
print("="*50)
print(f"COMPARATIVE ANALYSIS REPORT: {DATASET_NAME}")
print("="*50)
print("1. Effects of Outliers:")
print("   - In this clean dataset, there are no significant outliers.")
print("   - As a result, all three methods (MSE, MAE, Huber) produce very similar regression
lines.")
print(f"   - The slopes are nearly identical: MSE={mse_model.coef_[0]:.2f}, MAE={m_mae:.2f},
Huber={huber_model.coef_[0]:.2f}.")

print("\n2. Strengths and Weaknesses (Context of Clean Data):")
print("   - MSE: Works perfectly here. It is efficient and provides the Maximum Likelihood
Estimation for Gaussian noise.")
print("   - MAE: Also works well but is computationally heavier (requires iterative optimization).
No advantage over MSE here.")
```

```
print("   - Huber: Behaves like MSE because errors are small (within epsilon).")

print("\n3. Conclusion:")
print("   - For this clean dataset, MSE (Linear Regression) is the most suitable.")
print("   - Reason: It is the standard solution for normally distributed errors and is computationally simplest.")
print("="*50)
```

2. Code for Outliers Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, HuberRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error
from scipy.optimize import minimize

# --- Configuration ---
FILE_PATH = "salary_experience_with_outliers.csv"
DATASET_NAME = "Dataset with Outliers"

# --- Task 1: Data Exploration ---
print(f"--- Task 1: Data Exploration ({DATASET_NAME}) ---")
# Load dataset
df = pd.read_csv(FILE_PATH)
X = df['experience_years'].values
y = df['salary_k'].values

# Reshape X for sklearn (needs 2D array)
X_reshaped = X.reshape(-1, 1)

# Create scatter plot
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='orange', label='Data Points (with Outliers)', alpha=0.6)
plt.title(f"Salary vs Experience ({DATASET_NAME})")
plt.xlabel("Experience (Years)")
plt.ylabel("Salary (k)")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
plt.savefig('scatter_plot_outliers.png')
plt.close()
print("Scatter plot created. Visual inspection shows extreme outliers that deviate significantly from the main trend.\n")

# --- Task 2: Implement MSE Regression (Ordinary Least Squares) ---
print(f"--- Task 2: MSE Regression ({DATASET_NAME}) ---")
```

```
# Fit Linear Regression (minimizes MSE)
mse_model = LinearRegression()
mse_model.fit(X_reshaped, y)
y_pred_mse = mse_model.predict(X_reshaped)

# Calculate metrics
mse_mse = mean_squared_error(y, y_pred_mse)
mae_mse = mean_absolute_error(y, y_pred_mse)

print(f"MSE Model Coefficients: Slope={mse_model.coef_[0]:.2f},
Intercept={mse_model.intercept_:.2f}")
print(f"MSE (Mean Squared Error): {mse_mse:.2f}")
print(f"MAE (Mean Absolute Error): {mae_mse:.2f}")

# Plot MSE Regression
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='orange', label='Data Points', alpha=0.6)
plt.plot(X, y_pred_mse, color='red', linewidth=2, label='MSE Regression (OLS)')
plt.title(f"MSE Regression on {DATASET_NAME}")
plt.xlabel("Experience (Years)")
plt.ylabel("Salary (k)")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
plt.savefig('mse_regression_outliers.png')
plt.close()
print("\n")

# --- Task 3: Implement MAE Regression (Least Absolute Deviations) ---
print(f"--- Task 3: MAE Regression ({DATASET_NAME}) ---")

# Define MAE loss function for optimization
def mae_loss_func(params, X_vals, y_vals):
    m, c = params
    y_pred = m * X_vals + c
    return np.sum(np.abs(y_vals - y_pred))

# Optimize to find best m (slope) and c (intercept)
initial_guess = [1, 1]
result = minimize(mae_loss_func, initial_guess, args=(X, y))
m_mae, c_mae = result.x
y_pred_mae = m_mae * X + c_mae

# Calculate metrics for MAE model
mse_mae_model = mean_squared_error(y, y_pred_mae)
mae_mae_model = mean_absolute_error(y, y_pred_mae)

print(f"MAE Model Coefficients: Slope={m_mae:.2f}, Intercept={c_mae:.2f}")
```

```
print(f"MSE (Mean Squared Error): {mse_mae_model:.2f}")
print(f"MAE (Mean Absolute Error): {mae_mae_model:.2f}")

# Plot MAE Regression
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='orange', label='Data Points', alpha=0.6)
plt.plot(X, y_pred_mae, color='green', linewidth=2, label='MAE Regression (LAD)')
plt.title(f"MAE Regression on {DATASET_NAME}")
plt.xlabel("Experience (Years)")
plt.ylabel("Salary (k)")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
plt.savefig('mae_regression_outliers.png')
plt.close()
print("\n")

# --- Task 4: Implement Huber Regression ---
print(f"--- Task 4: Huber Regression ({DATASET_NAME}) ---")
# Fit Huber Regressor
# epsilon=1.35 is standard; smaller epsilon makes it more resistant to outliers (closer to MAE)
huber_model = HuberRegressor(epsilon=1.35)
huber_model.fit(X_reshaped, y)
y_pred_huber = huber_model.predict(X_reshaped)

# Calculate metrics
mse_huber = mean_squared_error(y, y_pred_huber)
mae_huber = mean_absolute_error(y, y_pred_huber)

print(f"Huber Model Coefficients: Slope={huber_model.coef_[0]:.2f},
Intercept={huber_model.intercept_:.2f}")
print(f"MSE (Mean Squared Error): {mse_huber:.2f}")
print(f"MAE (Mean Absolute Error): {mae_huber:.2f}")

# Plot All Comparisons
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='gray', label='Data Points', alpha=0.5)
plt.plot(X, y_pred_mse, color='red', linestyle='--', linewidth=2, label='MSE (OLS)')
plt.plot(X, y_pred_mae, color='green', linestyle='-.', linewidth=2, label='MAE (LAD)')
plt.plot(X, y_pred_huber, color='purple', linestyle='-', linewidth=2, label='Huber')
plt.title(f"Comparison of Loss Functions on {DATASET_NAME}")
plt.xlabel("Experience (Years)")
plt.ylabel("Salary (k)")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
plt.savefig('huber_regression_outliers.png')
plt.close()
```

```
print("\n")

# --- Task 5: Comparative Analysis Report ---
print("="*50)
print(f"COMPARATIVE ANALYSIS REPORT: {DATASET_NAME}")
print("="*50)
print("1. Effects of Outliers:")
print("   - The outliers significantly pull the MSE regression line towards them.")
print(f"   - MSE Slope: {mse_model.coef_[0]:.2f} (Distorted by outliers)")
print(f"   - MAE Slope: {m_mae:.2f} (Robust, ignores outliers)")
print(f"   - Huber Slope: {huber_model.coef_[0]:.2f} (Robust, similar to MAE)")

print("\n2. Strengths and Weaknesses (Context of Noisy Data):")
print("   - MSE: Highly sensitive to outliers. The squared term penalizes large errors heavily, causing the model to skew to accommodate anomalies.")
print("   - MAE: Very robust. It treats all deviations linearly, so a few extreme points don't shift the line much.")
print("   - Huber: A balanced approach. It is robust like MAE for large errors (linear loss) but differentiable at 0 like MSE (quadratic loss).")

print("\n3. Conclusion:")
print("   - For this noisy dataset, MAE or Huber Regression is most suitable.")
print("   - Reason: They provide a model that represents the majority of the data ('the trend') rather than trying to fit the anomalies.")
print("   - MSE is a poor choice here as it fails to capture the true underlying relationship due to the outliers.")
print("="*50)
```